

**UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ  
DEPARTAMENTO ACADÊMICO DE INFORMÁTICA  
CURSO SUPERIOR DE ENGENHARIA DE COMPUTAÇÃO**

**THIAGO CRISTOVÃO DE SOUZA**

**APLICAÇÃO MOBILE COM ARQUITETURA SERVERLESS PARA  
VALIDAÇÃO DE MÁQUINAS DE VENDAS**

**TRABALHO DE CONCLUSÃO DE CURSO DE GRADUAÇÃO**

**APUCARANA  
2026**

**THIAGO CRISTOVÃO DE SOUZA**

**APLICAÇÃO MOBILE COM ARQUITETURA SERVERLESS PARA  
VALIDAÇÃO DE MÁQUINAS DE VENDAS**

Trabalho de Conclusão de Curso de Graduação, apresentado à disciplina de Trabalho de Conclusão de Curso 1, do Curso Superior de Engenharia De Computação, da Universidade Tecnológica Federal do Paraná – Câmpus Apucarana, como requisito parcial para obtenção do título de Bacharel.

Orientador: Prof. Me. Muriel de Souza Godoi

**APUCARANA**

**2026**

## RESUMO

O crescimento das redes de vending machines impõe desafios operacionais relacionados à validação de equipamentos distribuídos geograficamente. Em arquiteturas tradicionais, esse processo tende a ser centralizado em um sistema monolítico, o que representa um ponto único de falha: a indisponibilidade do sistema central pode paralisar a operação em campo. O problema é agravado pelo fato de muitas máquinas operarem de forma offline, sem conectividade nativa, o que transfere ao aplicativo do operador a responsabilidade pela validação. Diante desse cenário, este trabalho propõe um ecossistema independente para validação de máquinas em redes de vending, baseado em arquitetura serverless na Google Cloud Platform, com o objetivo de oferecer resiliência operacional, escalabilidade e independência em relação à disponibilidade do sistema central. Como caso de estudo, adota-se o modelo Crane National 168, representativo das máquinas que operam de forma inteiramente offline, sem qualquer recurso de validação autônoma. O ecossistema proposto deverá ser composto por um serviço de validação em Go, executado no Google Cloud Functions; uma camada de exposição gerenciada pelo Google Cloud API Gateway; o envio assíncrono de mensagens via Google Cloud Pub/Sub; a persistência dos registros em Firestore; e uma aplicação mobile multiplataforma em Flutter, responsável pela leitura dos QR codes associados às máquinas. A comunicação assíncrona deverá assegurar que a validação prossiga mesmo durante indisponibilidades do sistema central, pois as mensagens permanecem enfileiradas até o restabelecimento do serviço. A infraestrutura será provisionada por meio de Infrastructure as Code com Terraform. Espera-se obter um sistema de validação desacoplado, resiliente e escalável, acompanhado de uma análise comparativa das decisões arquiteturais adotadas, contribuindo para a literatura sobre arquiteturas cloud-native aplicadas a cenários de operação distribuída.

**Palavras-chave:** Arquitetura Serverless. Computação em Nuvem. Aplicação Mobile. Mensageria Assíncrona. Vending Machine.

## ABSTRACT

The growth of vending machine networks poses operational challenges related to the validation of geographically distributed equipment. In traditional architectures, this process tends to be centralized in a monolithic system, which represents a single point of failure: the unavailability of the central system can halt field operations. The problem is compounded by the fact that many machines operate offline, without native connectivity, which shifts the responsibility for validation to the operator's application. Given this scenario, this work proposes an independent ecosystem for validating machines in vending networks, based on a serverless architecture on the Google Cloud Platform, aiming to provide operational resilience, scalability, and independence from the availability of the central system. As a case study, the Crane National 168 model is adopted, representative of machines that operate entirely offline, without any autonomous validation capability. The proposed ecosystem is expected to comprise a validation service written in Go, running on Google Cloud Functions; an exposure layer managed by Google Cloud API Gateway; asynchronous message delivery via Google Cloud Pub/Sub; record persistence in Firestore; and a cross-platform mobile application built in Flutter, responsible for reading the QR codes attached to the machines. Asynchronous communication is expected to ensure that validation proceeds even during outages of the central system, since messages remain queued until the service is restored. The infrastructure will be provisioned through Infrastructure as Code using Terraform. The expected result is a decoupled, resilient, and scalable validation system, accompanied by a comparative analysis of the architectural decisions adopted, contributing to the literature on cloud-native architectures applied to distributed operation scenarios.

**Keywords:** Serverless Architecture. Cloud Computing. Mobile Application. Asynchronous Messaging. Vending Machine.

## LISTA DE ABREVIATURAS E SIGLAS

### SIGLAS

|         |                                                |
|---------|------------------------------------------------|
| API     | Application Programming Interface              |
| ARPANET | Advanced Research Projects Agency Network      |
| BaaS    | Backend as a Service                           |
| CPU     | Central Processing Unit                        |
| EDA     | Event-Driven Architecture                      |
| FaaS    | Function as a Service                          |
| GCP     | Google Cloud Platform                          |
| HTTP    | Hypertext Transfer Protocol                    |
| IaaS    | Infrastructure as a Service                    |
| IaC     | Infrastructure as Code                         |
| IAM     | Identity and Access Management                 |
| IDE     | Integrated Development Environment             |
| IoT     | Internet of Things                             |
| JIT     | Just-In-Time                                   |
| JSON    | JavaScript Object Notation                     |
| JWT     | JSON Web Token                                 |
| NIST    | National Institute of Standards and Technology |
| NoSQL   | Not Only SQL                                   |
| PaaS    | Platform as a Service                          |
| QR      | Quick Response                                 |
| RFC     | Request for Comments                           |
| SaaS    | Software as a Service                          |
| TCC     | Trabalho de Conclusão de Curso                 |
| UTFPR   | Universidade Tecnológica Federal do Paraná     |
| VPS     | Virtual Private Server                         |

## SUMÁRIO

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>INTRODUÇÃO . . . . .</b>                                                     | <b>7</b>  |
| 1.1      | CONSIDERAÇÕES INICIAIS . . . . .                                                | 7         |
| 1.2      | OBJETIVOS . . . . .                                                             | 8         |
| 1.2.1    | Objetivo Geral . . . . .                                                        | 8         |
| 1.2.2    | Objetivos Específicos . . . . .                                                 | 8         |
| 1.3      | JUSTIFICATIVA . . . . .                                                         | 9         |
| 1.4      | ESTRUTURA DO TRABALHO . . . . .                                                 | 10        |
| <b>2</b> | <b>REFERENCIAL TEÓRICO . . . . .</b>                                            | <b>11</b> |
| 2.1      | VENDING MACHINES E OPERAÇÃO DISTRIBUÍDA . . . . .                               | 11        |
| 2.2      | COMPUTAÇÃO EM NUVEM . . . . .                                                   | 12        |
| 2.3      | ARQUITETURA SERVERLESS . . . . .                                                | 13        |
| 2.4      | ARQUITETURA ORIENTADA A EVENTOS E ENVIO DE MEN-<br>SAGENS ASSÍNCRONAS . . . . . | 14        |
| 2.5      | INFRASTRUCTURE AS CODE . . . . .                                                | 15        |
| 2.6      | DESENVOLVIMENTO MOBILE MULTIPLATAFORMA . . . . .                                | 15        |
| 2.7      | AUTENTICAÇÃO E CONTROLE DE ACESSO EM APIS . . . . .                             | 16        |
| 2.8      | LINGUAGENS COMPILADAS EM AMBIENTES SERVERLESS . .                               | 17        |
| <b>3</b> | <b>MATERIAIS E MÉTODO . . . . .</b>                                             | <b>18</b> |
| 3.1      | CARACTERIZAÇÃO DA PESQUISA . . . . .                                            | 18        |
| 3.2      | VISÃO GERAL DO FLUXO DA APLICAÇÃO . . . . .                                     | 18        |
| 3.3      | MATERIAIS . . . . .                                                             | 20        |
| 3.3.1    | Linguagem de Programação Go . . . . .                                           | 20        |
| 3.3.2    | Linguagem Dart e Framework Flutter . . . . .                                    | 21        |
| 3.3.3    | Google Cloud Platform . . . . .                                                 | 21        |
| 3.3.4    | Terraform . . . . .                                                             | 21        |
| 3.3.5    | Ambiente de Desenvolvimento Integrado . . . . .                                 | 22        |
| 3.3.6    | Controle de Versão . . . . .                                                    | 22        |
| 3.4      | MÉTODO . . . . .                                                                | 22        |
| 3.4.1    | Levantamento e Análise de Requisitos . . . . .                                  | 22        |
| 3.4.2    | Projeto da Arquitetura . . . . .                                                | 23        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 3.4.3    | Provisionamento da Infraestrutura . . . . .     | 23        |
| 3.4.4    | Implementação do Serviço de Validação . . . . . | 23        |
| 3.4.5    | Implementação da Aplicação Mobile . . . . .     | 23        |
| 3.4.6    | Integração e Testes End-to-End . . . . .        | 24        |
| 3.4.7    | Avaliação Técnica . . . . .                     | 24        |
| <b>4</b> | <b>RESULTADOS ESPERADOS . . . . .</b>           | <b>26</b> |
| <b>5</b> | <b>CRONOGRAMA . . . . .</b>                     | <b>28</b> |
|          | <b>REFERÊNCIAS . . . . .</b>                    | <b>29</b> |

## 1 INTRODUÇÃO

Esta seção apresenta o contexto e a motivação para o desenvolvimento do trabalho, delimitando o problema a ser abordado, os objetivos pretendidos, a justificativa das escolhas iniciais e a estrutura do documento.

### 1.1 CONSIDERAÇÕES INICIAIS

A expansão das redes de vending machines tem transformado a forma como produtos e serviços são distribuídos em ambientes públicos e corporativos. Nesse modelo de operação, equipes de campo são responsáveis por realizar visitas periódicas às máquinas para atividades de abastecimento, manutenção e coleta, necessitando validar cada equipamento de forma ágil e confiável. Em arquiteturas de software tradicionais, esse processo de validação tende a ser centralizado em um sistema monolítico, o que representa um ponto único de falha: qualquer indisponibilidade do sistema central pode impossibilitar a operação em campo, comprometendo a continuidade do serviço.

Soma-se a esse cenário um aspecto frequentemente negligenciado no setor de vending: boa parte das máquinas em operação no mercado opera tradicionalmente de forma offline, sem conectividade nativa com a internet (VendingExchange, 2025). Essas máquinas dependem exclusivamente do operador em campo para a coleta de dados, validação e sincronização de informações com o sistema central, o que torna ainda mais crítica a robustez da ferramenta utilizada por esse operador. Em situações de falha do sistema central, o operador fica sem qualquer mecanismo de validação confiável, dado que a própria máquina não dispõe de meios para confirmar a operação de forma autônoma.

Para delimitar o escopo prático do trabalho, pretende-se adotar como caso de estudo máquinas da categoria representada pelo modelo Crane National 168, fabricado pela Crane Merchandising Systems e amplamente utilizado em ambientes corporativos, escolas e locais de serviço. Trata-se de um equipamento snack de operação inteiramente offline, sem conectividade nativa com redes Wi-Fi ou celular, cuja interação se dá exclusivamente por meio do painel mecânico da própria máquina e dos módulos de pagamento embarcados. Essa classe de equipamento é representativa de uma realidade ainda predominante no mercado, na qual a máquina em si não participa do processo de validação remota — papel que recai inteiramente sobre o aplicativo utilizado pelo operador e sobre os sistemas de retaguarda.

Diante desse cenário, arquiteturas baseadas em computação em nuvem e no paradigma serverless apresentam-se como alternativas adequadas para o desenvolvimento de sistemas resilientes, escaláveis e de baixo custo operacional. Nesse modelo, a lógica de negócio pode ser fragmentada em funções independentes, executadas sob demanda e gerenciadas pela infraestrutura do provedor de nuvem, eliminando a necessidade de gerenciamento manual de servidores (ROBERTS, 2018). Em complemento, padrões de comunicação assíncrona baseados em filas de

mensagens podem permitir o desacoplamento entre componentes, contribuindo para que falhas em um serviço não se propaguem para os demais.

O presente trabalho propõe investigar o desenvolvimento de um ecossistema de validação de máquinas para redes de vending baseado nessas tecnologias, com a hipótese inicial de que ele seja composto por um serviço serverless hospedado na Google Cloud Platform, uma camada de envio de mensagens assíncronas via Google Cloud Pub/Sub e uma aplicação mobile multiplataforma desenvolvida em Flutter. A expectativa é que a aplicação mobile seja capaz de realizar a leitura de QR codes associados às máquinas e se comunicar com o serviço de validação de forma independente do sistema central, permitindo que operadores executem suas atividades em campo mesmo em cenários de instabilidade.

## 1.2 OBJETIVOS

Esta seção apresenta os objetivos que norteiam o desenvolvimento do trabalho.

### 1.2.1 Objetivo Geral

Investigar e propor uma abordagem para a validação remota de máquinas em redes de vending caracterizadas pela ausência de conectividade nativa, de forma a oferecer resiliência operacional, escalabilidade e independência em relação à disponibilidade do sistema central.

### 1.2.2 Objetivos Específicos

- Projetar a arquitetura cloud do ecossistema de validação, especificando os componentes, os fluxos de comunicação e as decisões de arquitetura consideradas;
- Avaliar a viabilidade da implementação do serviço de validação utilizando a linguagem Go sobre o Google Cloud Functions, com exposição via Google Cloud API Gateway e integração com o Google Cloud Pub/Sub para envio assíncrono de mensagens;
- Explorar o desenvolvimento da aplicação mobile em Flutter, com leitura de QR codes e comunicação autenticada com o serviço de validação por meio do Firebase Authentication;
- Definir uma estratégia inicial de persistência dos dados de validação, tendo o Firestore como banco de dados NoSQL gerenciado candidato;
- Estabelecer uma abordagem de provisionamento da infraestrutura cloud por meio de Infrastructure as Code utilizando Terraform, buscando reprodutibilidade e rastreabilidade do ambiente.

### 1.3 JUSTIFICATIVA

A escolha do tema se justifica pela relevância prática do problema abordado: a dependência de sistemas monolíticos centralizados para operações distribuídas em campo constitui uma limitação real e recorrente em ambientes de operação de vending machines. Esse problema é agravado pelo fato de que muitas máquinas em campo operam de forma offline, sem capacidade nativa de participar do processo de validação (VendingExchange, 2025), o que transfere integralmente a responsabilidade pela operação ao aplicativo do operador e aos sistemas de retaguarda. Quando esses sistemas se tornam indisponíveis, equipes inteiras podem ficar paralisadas. A adoção do modelo Crane National 168 como caso de estudo permite basear o trabalho em um cenário concreto e representativo do mercado, caracterizado por máquinas sem qualquer capacidade de validação autônoma, nas quais a robustez do aplicativo do operador é fator determinante para a continuidade do serviço. A proposta de desacoplar a lógica de validação em um serviço independente, comunicando-se de forma assíncrona, busca responder diretamente a esse problema por meio de uma abordagem arquitetural moderna e fundamentada na literatura de sistemas distribuídos.

Do ponto de vista tecnológico, a consideração da Google Cloud Platform como provedor de infraestrutura se justifica pela maturidade dos serviços disponíveis (Cloud Functions, API Gateway, Pub/Sub e Firestore) e pela integração nativa entre eles, características que podem reduzir a complexidade de configuração e facilitar a observabilidade do sistema. A possibilidade de uso do Terraform para gerenciamento da infraestrutura aproxima o projeto de práticas consolidadas de DevOps, contribuindo para que o ambiente seja reproduzível e auditável (HashiCorp, 2023).

A consideração da linguagem Go para a implementação do serviço de validação é motivada por dois fatores complementares: o desempenho favorável em ambientes serverless, dado que binários compilados tendem a apresentar tempos de inicialização significativamente menores em comparação a linguagens interpretadas (LEE; SATYAM; FOX, 2018), e o aprendizado de uma nova tecnologia com aplicabilidade direta no desenvolvimento de serviços cloud-native. O Flutter, por sua vez, é considerado para o desenvolvimento mobile pela possibilidade de geração de aplicações nativas para Android e iOS a partir de uma única base de código, o que pode reduzir o custo de manutenção e contribuir para a consistência de comportamento entre plataformas (Google LLC, 2023b).

O trabalho pretende contribuir academicamente ao documentar e analisar decisões arquiteturais de um sistema distribuído aplicado a um cenário concreto de operação em campo, podendo servir como referência para projetos similares que busquem migrar de arquiteturas monolíticas para ecossistemas desacoplados baseados em nuvem.

## 1.4 ESTRUTURA DO TRABALHO

O presente trabalho está organizado em seis seções. A Seção 1 apresenta o contexto, os objetivos, a justificativa e a estrutura do trabalho. A Seção 2 desenvolve o referencial teórico, abordando os conceitos de arquitetura serverless, computação em nuvem, mensageria assíncrona, Infrastructure as Code e desenvolvimento mobile multiplataforma. A Seção 3 descreve os materiais e métodos previstos, detalhando a stack tecnológica considerada, os requisitos do sistema e os procedimentos de desenvolvimento e avaliação. A Seção 4 apresenta os resultados esperados ao final do desenvolvimento. Por fim, a Seção 5 expõe o cronograma previsto para a execução do projeto.

## 2 REFERENCIAL TEÓRICO

Esta seção apresenta os fundamentos teóricos que embasam o desenvolvimento do trabalho. A estrutura parte do contexto de aplicação até os conceitos mais amplos de computação em nuvem, passando pelos paradigmas e padrões arquiteturais considerados para a solução proposta.

### 2.1 VENDING MACHINES E OPERAÇÃO DISTRIBUÍDA

Vending machines são pontos de venda automatizados que operam sem a necessidade de intervenção humana direta, dispensando produtos ou serviços mediante interação do consumidor com o equipamento (SOLANO *et al.*, 2017). Embora o conceito exista desde o século XIX, foi a partir da década de 1980 que as vending machines começaram a incorporar recursos de automação avançada, sendo considerada um marco histórico a máquina de Coca-Cola conectada à ARPANET na Carnegie Mellon University, em 1982, reconhecida como um dos primeiros dispositivos conectados em rede da história e precursor conceitual do que viria a ser denominado Internet das Coisas (IoT) (THETE *et al.*, 2024).

A operação de uma rede de vending machines envolve dois atores principais: o consumidor final, que interage com a máquina para adquirir produtos ou serviços, e o operador de campo, responsável pelas atividades de abastecimento, manutenção preventiva, coleta de valores e validação periódica dos equipamentos. Conforme apontado por Solano *et al.* (2017), o setor de vending é caracterizado por margens operacionais estreitas e alta fragmentação, com milhares de pequenos e médios operadores responsáveis por frotas distribuídas geograficamente, o que torna a eficiência das atividades de campo um fator crítico para a viabilidade econômica da operação.

A evolução tecnológica do setor levou ao surgimento das chamadas *smart vending machines*, que incorporam recursos como sensores embarcados, conectividade celular ou Wi-Fi, sistemas de pagamento cashless, telemetria em tempo real e integração com plataformas em nuvem para monitoramento remoto da frota (THETE *et al.*, 2024). Tais recursos viabilizam a transmissão de dados operacionais (níveis de estoque, status de componentes, registros de transações) para sistemas centrais, permitindo que operadores acompanhem o desempenho da frota de forma centralizada e tomem decisões baseadas em dados.

No entanto, uma parcela significativa do parque instalado de vending machines no mercado ainda opera de forma inteiramente offline, sem qualquer recurso de conectividade nativa (VendingExchange, 2025). Nessas máquinas, a interação ocorre exclusivamente por meio do painel mecânico e dos módulos de pagamento embarcados, e toda a coleta de dados operacionais depende da presença física do operador. Como caso de estudo deste trabalho, adota-se o modelo Crane National 168, fabricado pela Crane Merchandising Systems, representativo dessa categoria de equipamento. Trata-se de uma snack machine amplamente utilizada em ambientes

corporativos, escolas e locais de serviço, caracterizada pela ausência de conectividade com a internet e pela impossibilidade de participar diretamente de qualquer fluxo de validação remota, características que tornam o aplicativo do operador o único elo entre a máquina e os sistemas de retaguarda.

## 2.2 COMPUTAÇÃO EM NUVEM

A computação em nuvem refere-se ao fornecimento de recursos computacionais (servidores, armazenamento, bancos de dados, redes e software) por meio da internet, sob demanda e com modelo de pagamento baseado no uso efetivo (MELL; GRANCE, 2011). Seu surgimento está diretamente associado à convergência de tecnologias anteriores, como a virtualização de servidores, a computação em grade e a computação utilitária, e seu crescimento foi impulsionado pela necessidade de maior elasticidade e redução de custos de infraestrutura por parte de organizações de todos os portes (NAMASUDRA *et al.*, 2022).

O National Institute of Standards and Technology (NIST) define o modelo a partir de cinco características essenciais: o autosserviço sob demanda, que permite ao consumidor provisionar recursos computacionais sem interação humana com o provedor; o amplo acesso à rede, que disponibiliza os recursos por meio de mecanismos padrão sobre a internet; o compartilhamento de recursos (*resource pooling*), que permite ao provedor atender múltiplos consumidores a partir de uma mesma base física por meio de abstrações de virtualização; a elasticidade rápida, que possibilita o escalonamento ascendente ou descendente da capacidade conforme a demanda; e o serviço mensurável, que viabiliza a cobrança proporcional ao uso real dos recursos (MELL; GRANCE, 2011).

A organização dos serviços em nuvem é tradicionalmente descrita em três modelos principais. O *Infrastructure as a Service* (IaaS) oferece ao usuário recursos de infraestrutura virtualizados, como máquinas virtuais, redes e armazenamento, transferindo ao provedor a responsabilidade pela camada física. O *Platform as a Service* (PaaS) abstrai também o sistema operacional e o ambiente de execução, permitindo que o usuário concentre-se na implantação de aplicações. Por fim, o *Software as a Service* (SaaS) entrega aplicações prontas para uso por meio da internet, eliminando do usuário qualquer responsabilidade sobre a infraestrutura subjacente (MELL; GRANCE, 2011). À medida que os provedores de nuvem ampliam seus catálogos, novos modelos têm surgido como derivações desses três, incluindo o *Function as a Service* (FaaS) e o *Backend as a Service* (BaaS), discutidos na seção seguinte.

Quanto ao modelo de implantação, a computação em nuvem pode ser organizada em nuvens públicas, privadas, comunitárias e híbridas, variando conforme o controle exercido sobre os recursos e o público atendido (NAMASUDRA *et al.*, 2022). Nuvens públicas são operadas por provedores comerciais e disponibilizadas a qualquer interessado, oferecendo o maior grau de elasticidade e o menor custo inicial; nuvens privadas são restritas a uma única organização e oferecem maior controle sobre dados sensíveis; nuvens comunitárias são compartilhadas por

organizações com interesses comuns; e nuvens híbridas combinam dois ou mais modelos para atender requisitos heterogêneos. Os principais provedores comerciais de nuvem pública (Amazon Web Services, Microsoft Azure e Google Cloud Platform) oferecem catálogos amplos de serviços gerenciados que cobrem desde computação e armazenamento até bancos de dados, mensageria e ferramentas de observabilidade, viabilizando o desenvolvimento de aplicações distribuídas sem a necessidade de gestão direta da infraestrutura subjacente.

### 2.3 ARQUITETURA SERVERLESS

O paradigma serverless, apesar do nome, não implica a ausência de servidores, mas sim a abstração completa da sua gestão pelo provedor de nuvem. Nesse modelo, o desenvolvedor define unidades de lógica de negócio que são executadas sob demanda, escalonadas automaticamente conforme a carga e cobradas proporcionalmente ao tempo de execução e ao número de invocações (ROBERTS, 2018). Quando não há requisições, os custos tendem a ser reduzidos a níveis mínimos, característica conhecida como *scale-to-zero*, que diferencia o modelo serverless dos paradigmas anteriores de computação em nuvem.

O núcleo do paradigma serverless é o modelo *Function as a Service* (FaaS), no qual a unidade de implantação é uma função isolada que responde a eventos específicos, como requisições HTTP, mensagens em filas ou alterações em bancos de dados. Cada função é tipicamente curta, sem estado persistente entre invocações e executada em um ambiente efêmero gerenciado pelo provedor. Esse modelo é frequentemente complementado por serviços gerenciados de backend, como bancos de dados, sistemas de autenticação e mecanismos de envio de mensagens, configurando o que se convencionou chamar de *Backend as a Service* (BaaS) (ROBERTS, 2018). A combinação de FaaS e BaaS permite que aplicações inteiras sejam construídas com pouco ou nenhum código de infraestrutura escrito pelo desenvolvedor.

Entre as vantagens do modelo serverless destacam-se a eliminação do gerenciamento de servidores, a escalabilidade automática e horizontal, o modelo de custo baseado em execução e a simplificação do ciclo de *deploy*. Esses fatores tornam o paradigma particularmente adequado a cargas de trabalho com demanda irregular ou imprevisível, nas quais o provisionamento estático de servidores resultaria em desperdício de recursos em períodos de baixa utilização (EISMANN *et al.*, 2020).

Como limitações, o modelo apresenta o fenômeno de *cold start* (latência adicional na primeira invocação de uma função inativa), restrições quanto ao tempo máximo de execução por invocação e limitações de memória e CPU por instância (EISMANN *et al.*, 2020). O *cold start* é particularmente relevante em aplicações sensíveis à latência, uma vez que envolve a alocação do ambiente de execução, o carregamento do código da função e a inicialização do runtime, processo que pode adicionar centenas de milissegundos ao tempo total de resposta. Estratégias comuns de mitigação incluem a manutenção de instâncias aquecidas, o uso de linguagens compiladas com tempos de inicialização reduzidos, o pré-aquecimento programado de funções e o

suporte a concorrência por instância, que permite que múltiplas requisições sejam atendidas por uma mesma execução da função.

## 2.4 ARQUITETURA ORIENTADA A EVENTOS E ENVIO DE MENSAGENS ASSÍNCRONAS

A arquitetura orientada a eventos (*Event-Driven Architecture* — EDA) é um padrão arquitetural no qual os componentes de um sistema se comunicam por meio da produção e do consumo de eventos, de forma assíncrona e desacoplada (FOWLER, 2017). Nesse modelo, o produtor de um evento não possui conhecimento sobre os consumidores, o que reduz o acoplamento entre serviços e aumenta a resiliência do sistema como um todo. Conforme apontado por Laigner *et al.* (2024), a EDA tem ganhado adoção crescente no contexto de arquiteturas de microsserviços, dado que oferece um modelo de comunicação naturalmente alinhado à independência e à escalabilidade individual dos componentes.

A literatura distingue diferentes estilos de comunicação assíncrona orientada a eventos. No modelo de notificação simples (*event notification*), o produtor emite um aviso de que algo ocorreu, e os consumidores interessados podem reagir conforme suas próprias regras. No modelo de transferência de estado (*event-carried state transfer*), o próprio evento carrega os dados necessários para que os consumidores atuem sem precisar consultar o produtor. Já o *event sourcing* estabelece que o estado de um sistema seja derivado da sequência completa de eventos que ocorreram ao longo do tempo, permitindo reconstruir qualquer estado intermediário a partir do histórico (FOWLER, 2017). A escolha entre esses estilos depende do grau de acoplamento aceitável entre os componentes e dos requisitos de auditabilidade e rastreabilidade do sistema.

O envio assíncrono de mensagens em sistemas distribuídos é tipicamente realizado por meio de *message brokers*, componentes intermediários responsáveis por receber, armazenar e entregar mensagens entre produtores e consumidores. Dois modelos são predominantes: o modelo de fila (*point-to-point*), no qual cada mensagem é consumida por um único receptor, e o modelo *publish/subscribe*, no qual produtores publicam mensagens em tópicos e múltiplos consumidores podem recebê-las por meio de assinaturas independentes (LAIGNER *et al.*, 2024). Esse último modelo é particularmente adequado a cenários nos quais o mesmo evento pode interessar a diferentes componentes do sistema.

A principal contribuição desse padrão para sistemas distribuídos é a garantia de entrega: mesmo que um consumidor esteja temporariamente indisponível, as mensagens permanecem enfileiradas no *broker* e são processadas assim que o serviço receptor retorna à disponibilidade, eliminando o ponto único de falha característico de arquiteturas síncronas e contribuindo para o desacoplamento temporal entre os componentes. Aspectos complementares relevantes incluem a semântica de entrega (*at-most-once*, *at-least-once* ou *exactly-once*), a ordenação das mensagens dentro de uma mesma partição e as estratégias de tratamento de falhas, como filas

de mensagens mortas (*dead-letter queues*), que armazenam mensagens que não puderam ser processadas com sucesso após sucessivas tentativas (LAIGNER *et al.*, 2024).

## 2.5 INFRASTRUCTURE AS CODE

Infrastructure as Code (IaC) é a prática de gerenciar e provisionar infraestrutura de tecnologia da informação por meio de arquivos de configuração legíveis por máquina, em vez de processos manuais (MORRIS, 2016). Essa abordagem trata a infraestrutura da mesma forma que o código de software, permitindo versionamento, revisão por pares, testes automatizados e reprodutibilidade de ambientes. A adoção de IaC é frequentemente associada ao movimento DevOps, no qual busca-se reduzir as fronteiras entre as equipes de desenvolvimento e de operações por meio da automação e da padronização dos processos de entrega de software.

A literatura distingue duas abordagens principais de IaC: a declarativa e a imperativa. Na abordagem declarativa, o desenvolvedor descreve o estado desejado da infraestrutura (quais recursos devem existir e com quais propriedades) e a ferramenta é responsável por calcular e aplicar as mudanças necessárias para atingi-lo. Na abordagem imperativa, o desenvolvedor descreve uma sequência de passos a serem executados para alcançar o estado desejado (MORRIS, 2016). A abordagem declarativa tende a ser preferida em contextos de nuvem, dado que reduz a complexidade do raciocínio sobre o estado atual da infraestrutura e facilita operações idempotentes.

A adoção de IaC traz benefícios diretos para o ciclo de vida de aplicações em nuvem: a infraestrutura passa a ser auditável e replicável entre ambientes de desenvolvimento, homologação e produção, reduzindo a possibilidade de divergências e falhas associadas a configurações manuais (fenômeno conhecido como *configuration drift*). Outro benefício relevante é a possibilidade de tratar a infraestrutura como parte integrante do ciclo de revisão de código, permitindo que mudanças sejam discutidas, testadas e aprovadas antes de serem aplicadas em produção.

Entre as ferramentas que se enquadram nessa abordagem, o Terraform, desenvolvido pela HashiCorp, destaca-se como uma opção declarativa, de código aberto e agnóstica de provedor, sendo considerada para o gerenciamento da infraestrutura cloud deste trabalho (HashiCorp, 2023).

## 2.6 DESENVOLVIMENTO MOBILE MULTIPLATAFORMA

O desenvolvimento mobile multiplataforma busca permitir que uma única base de código gere aplicações executáveis em diferentes sistemas operacionais, tipicamente Android e iOS, reduzindo o esforço de manutenção e o tempo de entrega quando comparado ao desenvolvimento nativo separado para cada plataforma. As principais abordagens contemporâneas podem ser classificadas em três categorias: aplicações híbridas, baseadas em tecnologias web

encapsuladas em um *wrapper* nativo; aplicações cross-platform com renderização nativa, que utilizam componentes da plataforma alvo; e aplicações com renderização própria, nas quais o framework controla diretamente o desenho da interface por meio de um motor gráfico independente das APIs nativas.

Cada uma dessas abordagens apresenta compromissos próprios. Aplicações híbridas tendem a oferecer o ciclo de desenvolvimento mais rápido, ao custo de menor desempenho e de limitações no acesso a recursos nativos do dispositivo. Aplicações cross-platform com renderização nativa oferecem aparência consistente com cada plataforma, mas dependem da estabilidade das APIs nativas subjacentes. Aplicações com renderização própria proporcionam o controle mais granular sobre a interface e o comportamento da aplicação, viabilizando uma experiência visual idêntica entre plataformas, com o custo de não herdarem automaticamente atualizações visuais dos sistemas operacionais.

Um framework representativo da abordagem de renderização própria é o Flutter, criado pelo Google, que utiliza a linguagem Dart e o motor gráfico Skia para gerar interfaces consistentes entre Android e iOS (Google LLC, 2023b), sendo cogitado como candidato ao desenvolvimento da aplicação mobile deste trabalho.

## 2.7 AUTENTICAÇÃO E CONTROLE DE ACESSO EM APIS

A autenticação e o controle de acesso são aspectos centrais no projeto de APIs expostas publicamente, especialmente em arquiteturas distribuídas nas quais a comunicação ocorre entre clientes externos e serviços hospedados em nuvem. Conceitualmente, a autenticação refere-se ao processo de verificar a identidade de uma entidade (usuário ou sistema), enquanto a autorização refere-se ao processo de determinar quais ações essa entidade está autorizada a executar. Embora frequentemente tratadas em conjunto, são preocupações distintas e tipicamente implementadas por mecanismos separados.

Um padrão amplamente adotado para autenticação em APIs é o uso de tokens de acesso, mecanismo no qual o cliente, após uma etapa inicial de autenticação com um provedor de identidade, recebe um token que deve ser incluído nas requisições subsequentes para comprovar sua identidade. Esse modelo apresenta vantagens em relação à autenticação baseada em credenciais a cada requisição, dado que o token pode ter tempo de vida limitado, escopo restrito e ser revogado independentemente das credenciais originais do usuário.

O JSON Web Token (JWT), padrão definido pela RFC 7519, é uma das implementações mais difundidas desse mecanismo (JONES; BRADLEY; SAKIMURA, 2015). Um JWT consiste em um objeto JSON assinado criptograficamente, composto por um cabeçalho, um conjunto de *claims* (informações sobre o portador do token) e uma assinatura digital. Sua verificação não requer consulta ao emissor a cada requisição, característica que o torna particularmente adequado a arquiteturas distribuídas e serviços *stateless*, nas quais a manutenção de sessões em memória seria inviável ou indesejável.

Em sistemas serverless, é comum delegar a etapa de validação do token a uma camada intermediária, frequentemente um API Gateway, que valida a assinatura e o conteúdo do token antes de encaminhar a requisição ao serviço de backend, evitando que cada função precise re-implementar essa lógica. Essa abordagem reforça o princípio de separação de responsabilidades e reduz a superfície de exposição dos serviços internos.

Nesse contexto, o Firebase Authentication, serviço gerenciado pelo Google, oferece emissão de JWTs e suporte a múltiplos provedores de identidade, sendo um candidato natural à camada de autenticação deste trabalho.

## 2.8 LINGUAGENS COMPILADAS EM AMBIENTES SERVERLESS

A escolha da linguagem de programação em ambientes serverless impacta diretamente o desempenho e o custo da solução, uma vez que o tempo de execução é fator direto na cobrança e o tempo de inicialização influencia a latência percebida pelo usuário. Linguagens interpretadas, como Python e JavaScript, oferecem ciclos de desenvolvimento ágeis e ampla disponibilidade de bibliotecas, mas costumam apresentar tempos de *cold start* mais altos, dado que o ambiente de execução precisa carregar o interpretador e os módulos da aplicação antes de executar o código de fato (LEE; SATYAM; FOX, 2018). Linguagens com máquinas virtuais robustas, como Java e C#, oferecem desempenho elevado em regime estável, mas tipicamente apresentam tempos de inicialização ainda maiores devido à carga inicial da máquina virtual e ao processo de compilação *just-in-time* (JIT).

Linguagens compiladas para código de máquina, por sua vez, produzem binários que podem ser carregados diretamente pelo sistema operacional, reduzindo significativamente o tempo de inicialização da função e oferecendo desempenho consistente desde a primeira invocação (LEE; SATYAM; FOX, 2018). Esse comportamento é particularmente vantajoso em cenários nos quais a função é invocada de forma esporádica e o *cold start* representa parte significativa do tempo total de resposta.

Outro fator relevante é o modelo de concorrência da linguagem, especialmente em cenários nos quais a função precisa realizar múltiplas operações de entrada e saída de forma eficiente, como o atendimento de uma requisição HTTP simultaneamente ao envio de uma mensagem para um *message broker*. Linguagens com suporte nativo a concorrência leve, baseada em *green threads* ou modelos similares, oferecem vantagens nesse tipo de cenário ao permitir o processamento paralelo sem o custo de *threads* do sistema operacional.

Como representante dessas características, a linguagem Go combina compilação para código de máquina, tipagem estática e suporte nativo a concorrência por meio de goroutines (Google LLC, 2023c), motivos pelos quais é considerada para a implementação do serviço de validação deste trabalho.

### 3 MATERIAIS E MÉTODO

Esta seção apresenta os materiais e o método que orientam o desenvolvimento do trabalho. A primeira parte descreve as tecnologias, ferramentas e ambientes considerados ao longo do projeto; a segunda detalha o método previsto, organizado em fases sequenciais que cobrem desde o levantamento de requisitos até a avaliação da solução proposta.

#### 3.1 CARACTERIZAÇÃO DA PESQUISA

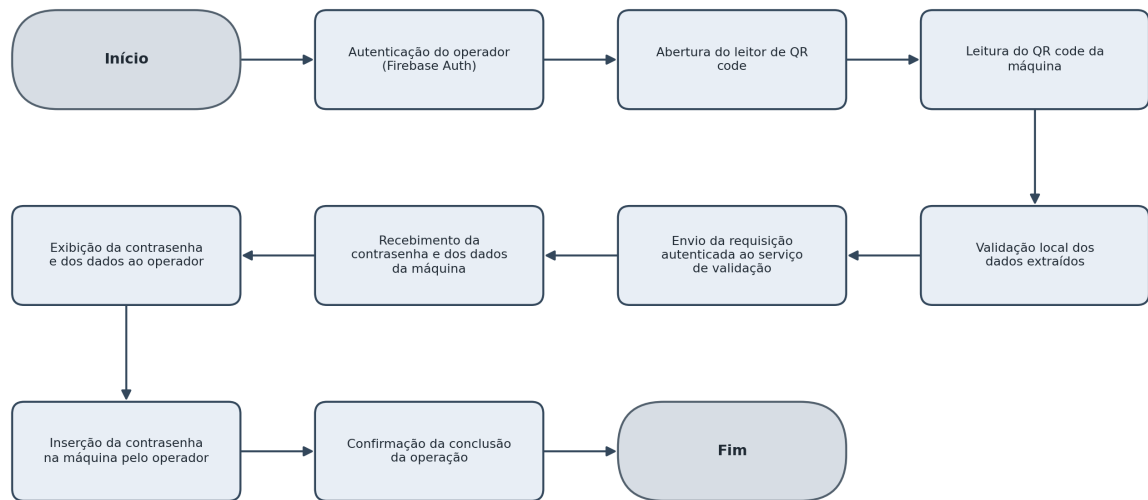
Este trabalho caracteriza-se como uma pesquisa aplicada, orientada para a solução de um problema prático e para a geração de conhecimento com aplicabilidade direta. O foco reside na concepção e na avaliação de um ecossistema de validação de máquinas em redes de vending baseado em arquitetura serverless, considerando como caso de estudo o modelo Crane Nacional 168, representativo das máquinas que operam de forma offline e sem qualquer recurso de integração nativa com a internet.

O desenvolvimento será conduzido sob os princípios da Engenharia de Software, adotando uma abordagem estruturada que abrange o ciclo de vida completo do projeto: o levantamento e a análise de requisitos funcionais e não funcionais, o desenho da arquitetura cloud, a implementação do serviço de validação e da aplicação mobile, a realização de testes unitários e de integração e a avaliação técnica da solução. Essa abordagem busca contribuir para a qualidade, a manutenibilidade e a reprodutibilidade do artefato a ser produzido.

#### 3.2 VISÃO GERAL DO FLUXO DA APLICAÇÃO

Antes da apresentação detalhada das tecnologias e do método, esta seção descreve o fluxo de operação previsto para a aplicação mobile utilizada pelo operador em campo. Esse fluxo orienta tanto a definição dos requisitos funcionais quanto a estruturação das telas e dos contratos de comunicação entre os componentes do sistema.

A Figura 1 ilustra o fluxo previsto, da autenticação do operador até a confirmação da conclusão da operação. O fluxo deverá iniciar com a autenticação do operador, seguida da abertura do leitor de QR code e da leitura do código fixado na máquina. Após uma validação local dos dados extraídos, a aplicação deverá enviar uma requisição autenticada ao serviço de validação e, na resposta, receber a contrasenha, que será exibida ao operador juntamente com os dados da máquina. O operador deverá, então, digitar a contrasenha na máquina e confirmar a conclusão da operação no aplicativo. Vale observar que a comunicação com o serviço de validação deverá ocorrer de forma independente do sistema central: caso este esteja indisponível, o fluxo do operador não será interrompido.

**Figura 1 – Fluxo previsto de funcionamento da aplicação mobile.****Fonte: Autoria própria.**

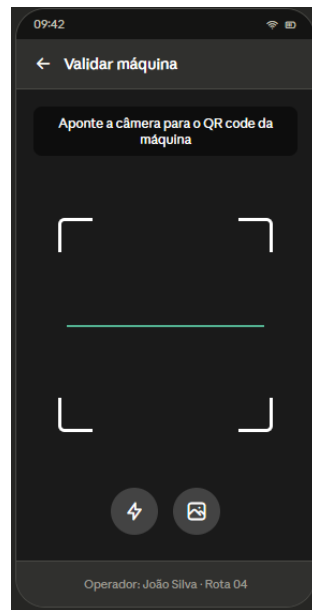
As duas telas principais do aplicativo, esboçadas em caráter preliminar, são apresentadas na Figura 2. A tela de leitura (Figura 2a) prevê que o operador aponte a câmera do dispositivo para o código fixado na máquina; ao reconhecer um QR code válido, a aplicação deverá extrair os dados nele codificados e disparar a requisição de validação. A tela de resultado (Figura 2b), por sua vez, deverá ser exibida após a resposta do serviço, destacando a contrasenha gerada, os dados retornados sobre a máquina e o botão de confirmação, por meio do qual o operador registrará a conclusão da operação.

Para fins de delimitação do escopo, considera-se que o QR code fixado em cada máquina carregue um conjunto mínimo de informações que permita a validação remota sem depender de consulta a sistemas externos durante a leitura. A título de exemplo, esse conjunto poderia contemplar o identificador único da máquina, o modelo do equipamento, o identificador da localização física e um token de validação gerado periodicamente. Após a validação bem-sucedida, o sistema poderia armazenar registros contendo o identificador da máquina, o identificador do operador, o horário da validação, a contrasenha gerada, leituras operacionais informadas pelo operador (como o valor do medidor interno e o número de unidades vendidas desde a última coleta) e o status de confirmação da operação. A definição final dos campos transportados pelo QR code e dos registros persistidos será detalhada durante a fase de levantamento de requisitos.

Tanto o fluxo apresentado quanto os esboços das telas representam uma proposta inicial, sujeita a refinamentos durante as fases de levantamento de requisitos e de implementação. O objetivo, nesta etapa, é estabelecer um ponto de partida visual e conceitual que oriente as decisões subsequentes.

**Figura 2 – Esboços preliminares das telas principais da aplicação mobile.**

**(a) Tela de leitura do QR code.**



**(b) Tela de resultado da validação.**



**Fonte: Autoria própria.**

### 3.3 MATERIAIS

Nesta seção são apresentadas as principais ferramentas, tecnologias e ambientes de desenvolvimento considerados para a concepção, a implementação e a avaliação do ecossistema proposto. As escolhas a seguir são tratadas como alternativas iniciais, motivadas pela adequação aos requisitos do projeto, pela maturidade no mercado e pela integração entre os componentes do ecossistema da Google Cloud Platform, podendo ser ajustadas conforme as necessidades identificadas durante o desenvolvimento.

#### 3.3.1 Linguagem de Programação Go

A linguagem Go foi selecionada preliminarmente para a implementação do serviço de validação. Ela é compilada, estaticamente tipada e com suporte nativo à concorrência por meio de goroutines, desenvolvida pelo Google (Google LLC, 2023c). Suas características principais (compilação rápida, baixo consumo de memória e geração de binários autônomos) tendem a torná-la adequada ao contexto serverless, no qual o tempo de inicialização da função impacta diretamente a latência percebida pelo usuário (LEE; SATYAM; FOX, 2018).

### 3.3.2 Linguagem Dart e Framework Flutter

Para a construção da aplicação mobile, considera-se inicialmente o Flutter, framework de código aberto desenvolvido pelo Google, que permite o desenvolvimento de aplicações nativamente compiladas para Android e iOS a partir de uma única base de código escrita na linguagem Dart (Google LLC, 2023b). Sua arquitetura baseada em *widgets* reativos e o suporte a recursos como Hot Reload podem acelerar o ciclo de desenvolvimento e iteração. Para a leitura dos QR codes, uma possível alternativa é o pacote `mobile_scanner`, que oferece integração com a câmera dos dispositivos e suporte aos formatos considerados no trabalho.

### 3.3.3 Google Cloud Platform

A Google Cloud Platform (GCP) é considerada como provedor de infraestrutura cloud do projeto. Os serviços gerenciados inicialmente considerados são:

- **Google Cloud Functions (2ª geração):** possível ambiente serverless de execução do serviço de validação, baseado em Cloud Run, com suporte a concorrência por instância e mitigação do problema de *cold start*;
- **Google Cloud API Gateway:** candidato a camada de exposição do serviço de validação, responsável pela autenticação das requisições, pelo controle de acesso e pelo versionamento dos endpoints;
- **Google Cloud Pub/Sub:** serviço de envio de mensagens assíncronas baseado no modelo *publish/subscribe*, considerado para desacoplar o serviço de validação do sistema central (Google LLC, 2023a);
- **Firestore:** banco de dados NoSQL gerenciado, considerado para a persistência dos registros de validação;
- **Firebase Authentication:** possível serviço de autenticação, responsável pela emissão de JSON Web Tokens (JWT) que autenticariam as requisições entre a aplicação mobile e o API Gateway (JONES; BRADLEY; SAKIMURA, 2015);
- **Google Cloud Logging e Cloud Monitoring:** ferramentas nativas de observabilidade que poderiam ser utilizadas para o registro de logs e o monitoramento das funções e dos fluxos de mensagens.

### 3.3.4 Terraform

Para o provisionamento e o gerenciamento da infraestrutura cloud, foi selecionado preliminarmente o Terraform, ferramenta de Infrastructure as Code desenvolvida pela HashiCorp

(HashiCorp, 2023). Sua adoção poderia contribuir para que o ambiente seja reproduzível, versionável e auditável, alinhando o projeto às práticas modernas de DevOps (MORRIS, 2016).

### 3.3.5 Ambiente de Desenvolvimento Integrado

Para a codificação, a depuração e o gerenciamento do projeto, considera-se inicialmente o Visual Studio Code (VS Code), editor leve e extensível com suporte às linguagens Go e Dart por meio de extensões oficiais. Recursos como autocompletar inteligente, depurador integrado, controle de versão Git embutido e terminal integrado tendem a aumentar a produtividade do desenvolvimento.

### 3.3.6 Controle de Versão

O Git é considerado como sistema de controle de versão distribuído, com possível hospedagem dos repositórios no GitHub. O versionamento seria aplicado tanto ao código-fonte do serviço de validação e da aplicação mobile quanto aos arquivos de configuração da infraestrutura escritos em Terraform, contribuindo para o rastreamento das mudanças em todas as camadas do projeto.

## 3.4 MÉTODO

O método de desenvolvimento previsto segue uma abordagem incremental, organizada em fases sequenciais que cobrem o ciclo de vida do projeto. As atividades estão fundamentadas nos princípios da Engenharia de Software descritos por Pressman (2009) e adaptadas ao contexto de sistemas distribuídos cloud-native discutido por Coulouris *et al.* (2013). As fases são apresentadas a seguir.

### 3.4.1 Levantamento e Análise de Requisitos

A primeira fase prevê o levantamento e a análise dos requisitos funcionais e não funcionais do sistema. Os requisitos funcionais deverão ser definidos a partir do fluxo de operação ilustrado na Figura 1, abrangendo a leitura de QR codes, a comunicação com o serviço de validação, a geração da contrasenha, o envio assíncrono de mensagens ao sistema central e a persistência dos registros de validação. Os requisitos não funcionais deverão contemplar aspectos como resiliência, escalabilidade, segurança, observabilidade e tempo de resposta. Esta fase deverá produzir um documento consolidado de requisitos que sirva como insumo para as fases subsequentes.

### 3.4.2 Projeto da Arquitetura

A segunda fase deverá compreender o detalhamento da arquitetura do ecossistema, incluindo a definição dos componentes, dos contratos de comunicação entre eles, do modelo de dados do banco escolhido, do esquema do payload do QR code e dos tópicos do serviço de mensageria. Nesta fase também deverá ser especificada a estratégia de autenticação e autorização entre a aplicação mobile, o API Gateway e o serviço de validação, bem como o tratamento de erros e os fluxos de retentativa em caso de falhas. O artefato resultante deverá ser o documento de arquitetura, contendo diagramas de componentes, diagramas de sequência e o modelo de dados. As telas esboçadas na Figura 2 poderão ser refinadas nesta etapa, incorporando ajustes de usabilidade identificados durante o detalhamento da arquitetura.

### 3.4.3 Provisionamento da Infraestrutura

A terceira fase deverá consistir na escrita dos arquivos de Infrastructure as Code que descrevem a infraestrutura cloud do projeto. Pretende-se provisionar os recursos necessários para a execução do sistema, incluindo o ambiente serverless de execução, a camada de exposição da API, o serviço de mensageria, o banco de dados gerenciado e o provedor de autenticação, além das configurações de IAM, redes e logs. A infraestrutura deverá ser versionada em repositório dedicado e validada em um ambiente de desenvolvimento antes de qualquer aplicação em produção.

### 3.4.4 Implementação do Serviço de Validação

A quarta fase deverá abranger a implementação do serviço de validação. Considera-se a adoção de princípios de arquitetura limpa, com separação entre as camadas de transporte (handler HTTP), aplicação (caso de uso de validação), domínio (regras de validação do QR code) e infraestrutura (acesso ao banco e ao serviço de mensageria). Pretende-se escrever testes unitários para as regras de domínio e testes de integração para os fluxos completos da função.

### 3.4.5 Implementação da Aplicação Mobile

A quinta fase deverá tratar da implementação da aplicação mobile, estruturada de acordo com padrões de arquitetura usuais no ecossistema escolhido. Ela deverá contemplar a leitura do QR code, a validação local dos dados extraídos, a autenticação do operador, o envio das informações ao serviço de validação e a exibição da contrasenha e dos dados da máquina ao usuário, seguindo o fluxo previsto na Figura 1. Pretende-se escrever testes automatizados para os principais fluxos da interface.

### 3.4.6 Integração e Testes End-to-End

A sexta fase deverá abranger a integração entre os componentes desenvolvidos e a realização de testes end-to-end que validem o fluxo completo do sistema, desde a leitura do QR code até o consumo da mensagem pelo sistema central simulado. Cenários de falha poderão ser deliberadamente induzidos (indisponibilidade do consumidor de mensagens, falha de rede no dispositivo móvel, erros de autenticação) para verificar o comportamento de resiliência da arquitetura.

### 3.4.7 Avaliação Técnica

A sétima e última fase deverá abranger a avaliação técnica do sistema, organizada em torno dos três atributos de qualidade que fundamentam o objetivo geral do trabalho. Para cada atributo, são previstos métodos específicos de verificação, descritos a seguir.

- **Resiliência operacional:** avaliação por meio de testes de injeção de falhas, nos quais cenários adversos serão deliberadamente induzidos no ambiente de execução. Os cenários previstos incluem a indisponibilidade temporária do consumidor de mensagens, falhas intermitentes de rede no dispositivo móvel e indisponibilidade do provedor de autenticação. A verificação consistirá em confirmar que as operações em andamento não sejam perdidas, que as mensagens permaneçam enfileiradas durante a indisponibilidade dos consumidores e sejam processadas corretamente após o restabelecimento do serviço, e que o operador em campo receba feedback adequado em cada cenário;
- **Escalabilidade:** avaliação por meio de testes de carga simulando diferentes volumes simultâneos de operações de validação. As métricas observadas incluem o tempo de resposta médio e percentis (p50, p95 e p99), o comportamento do serviço sob picos de demanda, a quantidade de instâncias paralelas instanciadas pela plataforma serverless e a estabilidade da latência fim a fim conforme a carga cresce. Esses dados serão coletados por meio das ferramentas de observabilidade do provedor de nuvem;
- **Independência em relação à disponibilidade do sistema central:** avaliação por meio de cenários nos quais o sistema central é colocado intencionalmente em estado de indisponibilidade durante a execução de operações de validação. A verificação consistirá em confirmar que as operações continuem sendo processadas pelo serviço de validação sem interrupção perceptível ao operador, que as mensagens geradas durante o período de indisponibilidade permaneçam preservadas na camada de mensageria, e que sejam consumidas e refletidas no sistema central após o seu restabelecimento, sem perda de informações nem necessidade de intervenção manual.

A avaliação deverá ser conduzida em ambiente de desenvolvimento e não envolverá usuários finais, concentrando-se em métricas técnicas obtidas por meio das ferramentas de observabilidade do provedor de nuvem. Complementarmente, prevê-se uma análise comparativa qualitativa entre a abordagem proposta e a abordagem monolítica tradicional, discutindo de forma crítica as decisões arquiteturais adotadas e os trade-offs envolvidos.

## 4 RESULTADOS ESPERADOS

Esta seção apresenta os resultados esperados ao final do desenvolvimento do trabalho, considerando o escopo definido nas seções anteriores e as fases previstas no método descrito na Seção 3.

Ao final do ciclo de desenvolvimento, espera-se obter um ecossistema funcional de validação de máquinas para redes de vending, composto por três artefatos principais e interligados: um serviço de validação executado em ambiente serverless na nuvem pública, uma aplicação mobile multiplataforma utilizada pelo operador em campo e uma infraestrutura cloud gerenciada por Infrastructure as Code. Esses artefatos deverão operar de forma integrada, atendendo aos atributos de qualidade que fundamentam o objetivo geral do trabalho: resiliência operacional, escalabilidade e independência em relação à disponibilidade do sistema central.

A aplicação mobile deverá oferecer ao operador em campo uma interface clara e objetiva, capaz de realizar a leitura do QR code fixado em cada máquina, validar localmente os dados extraídos, autenticar a requisição junto ao serviço de validação e exibir a contrasenha gerada para inserção na máquina. A experiência de uso deverá seguir o fluxo previsto na Figura 1 e contemplar as telas esboçadas nas Figuras 2a e 2b, refinadas durante as fases de levantamento de requisitos e de implementação. Espera-se que a aplicação permaneça funcional mesmo diante de cenários de instabilidade do sistema central, preservando a continuidade da operação em campo.

O serviço de validação, hospedado em ambiente serverless, deverá ser capaz de processar as requisições da aplicação mobile, gerar contrasenhhas válidas, persistir os registros de validação no banco de dados gerenciado e despachar mensagens assíncronas para o sistema central por meio da camada de mensageria. Espera-se que o serviço apresente tempos de resposta adequados ao uso em campo, tanto em cenários de *cold start* quanto em regime de operação contínua, e que escale automaticamente conforme a demanda gerada por múltiplos operadores em paralelo.

A infraestrutura cloud, descrita inteiramente por meio de arquivos de Infrastructure as Code, deverá permitir o provisionamento reproduzível do ambiente em diferentes estágios (desenvolvimento, homologação e produção), com configurações de rede, identidade e observabilidade devidamente versionadas. Esse artefato deverá servir também como documentação executável da arquitetura proposta, facilitando futuras evoluções e auditorias.

Além dos artefatos técnicos, espera-se como resultado uma análise comparativa qualitativa entre a abordagem serverless proposta e a abordagem monolítica tradicional, discutindo de forma crítica as decisões arquiteturais adotadas e os trade-offs envolvidos. Os resultados dos testes de injeção de falhas, dos testes de carga e dos cenários de indisponibilidade do sistema central deverão ser consolidados e apresentados, permitindo avaliar em que medida os atributos de qualidade previstos foram alcançados.

Cabe ressaltar que este trabalho não pretende propor uma solução comercial pronta para

implantação em produção, tampouco substituir sistemas existentes em operadores específicos do setor de vending. O escopo está delimitado à concepção, à implementação e à avaliação de um ecossistema independente, em ambiente de desenvolvimento, com finalidade acadêmica. Aspectos como integração com sistemas legados específicos, suporte a múltiplos modelos de máquinas, processamento de pagamentos e gestão completa de frotas estão fora do escopo deste trabalho, embora a arquitetura proposta possa servir como ponto de partida para evoluções futuras nessas direções.

Espera-se, por fim, que o trabalho contribua academicamente ao documentar e analisar decisões arquiteturais de um sistema distribuído event-driven aplicado a um cenário concreto de operação em campo, servindo como referência para projetos similares que busquem migrar de arquiteturas monolíticas para ecossistemas desacoplados baseados em nuvem.

## 5 CRONOGRAMA

O cronograma estimado para a execução do projeto é apresentado na Figura 3, organizado em onze etapas distribuídas entre março e dezembro. As etapas 1 e 2 correspondem ao trabalho já desenvolvido durante o TCC 1, abrangendo a revisão bibliográfica que fundamenta o referencial teórico e a elaboração e defesa da proposta do trabalho.

As etapas 3 a 9 seguem as fases do método descrito na Seção 3.4: o levantamento e a análise de requisitos, o projeto da arquitetura, o provisionamento da infraestrutura, a implementação do serviço de validação, a implementação da aplicação mobile, a integração e os testes end-to-end e, por fim, a avaliação técnica da solução.

A etapa 10, de redação do TCC, concentra-se entre setembro e novembro e ocorre em paralelo às etapas de implementação e avaliação, de modo que o texto evolua junto com o desenvolvimento. A etapa 11 prevê a entrega da versão final à banca em novembro e a defesa no início de dezembro.

**Figura 3 – Cronograma estimado das etapas do projeto.**

| Etapa                                      | MAR | ABR | MAI | JUN | JUL | AGO | SET | OUT | NOV | DEZ |
|--------------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1 - Revisão Bibliográfica                  | ■   | ■   | ■   |     |     |     |     |     |     |     |
| 2 - Elaboração e Defesa da Proposta (TCC1) |     | ■   | ■   | ■   |     |     |     |     |     |     |
| 3 - Levantamento e Análise de Requisitos   |     |     |     | ■   | ■   |     |     |     |     |     |
| 4 - Projeto da Arquitetura                 |     |     |     |     | ■   | ■   |     |     |     |     |
| 5 - Provisionamento da Infraestrutura      |     |     |     |     |     | ■   |     |     |     |     |
| 6 - Implementação do Serviço de Validação  |     |     |     |     |     | ■   | ■   |     |     |     |
| 7 - Implementação da Aplicação Mobile      |     |     |     |     |     |     | ■   | ■   |     |     |
| 8 - Integração e Testes End-to-End         |     |     |     |     |     |     |     | ■   |     |     |
| 9 - Avaliação Técnica                      |     |     |     |     |     |     |     | ■   | ■   |     |
| 10 - Redação do TCC                        |     |     |     |     |     |     | ■   | ■   | ■   |     |
| 11 - Entrega à Banca e Defesa              |     |     |     |     |     |     |     |     | ■   | ■   |

**Fonte: Autoria própria.**

## REFERÊNCIAS

- COULOURIS, George *et al.* **Sistemas Distribuídos - 5ed**: Conceitos e projeto. Bookman Editora, 2013. ISBN 9788582600542. Disponível em: <https://books.google.com.br/books?id=6WU3AgAAQBAJ>. Citado na página 22.
- EISMANN, Simon *et al.* Serverless applications: Why, when, and how? **IEEE Software**, IEEE, v. 38, n. 1, p. 32–39, 2020. Citado na página 13.
- FOWLER, Martin. **What do you mean by “Event-Driven”?** 2017. Martinfowler.com. Disponível em: <https://martinfowler.com/articles/201701-event-driven.html>. Acesso em: 20 abr. 2025. Citado na página 14.
- Google LLC. **Cloud Pub/Sub Documentation**. [S.l.], 2023. Disponível em: <https://cloud.google.com/pubsub/docs/overview>. Acesso em: 20 abr. 2025. Citado na página 21.
- \_\_\_\_\_. **Flutter Documentation**. [S.l.], 2023. Disponível em: <https://docs.flutter.dev>. Acesso em: 20 abr. 2025. Citado 3 vezes nas páginas 9, 16 e 21.
- \_\_\_\_\_. **The Go Programming Language Documentation**. [S.l.], 2023. Disponível em: <https://go.dev/doc/>. Acesso em: 20 abr. 2025. Citado 2 vezes nas páginas 17 e 20.
- HashiCorp. **Terraform Documentation**. [S.l.], 2023. Disponível em: <https://developer.hashicorp.com/terraform/docs>. Acesso em: 20 abr. 2025. Citado 3 vezes nas páginas 9, 15 e 22.
- JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. **RFC 7519**: JSON Web Token (JWT). 2015. Internet Engineering Task Force (IETF). Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Acesso em: 20 abr. 2025. Citado 2 vezes nas páginas 16 e 21.
- LAIGNER, Rodrigo *et al.* Data management in microservices: State of the practice, challenges, and research directions. **Proceedings of the VLDB Endowment**, VLDB Endowment, v. 17, n. 3, p. 379–392, 2024. Disponível em: <https://www.vldb.org/pvldb/vol17/p379-laigner.pdf>. Citado 2 vezes nas páginas 14 e 15.
- LEE, Hyungro; SATYAM, Kumar; FOX, Geoffrey. Evaluation of production serverless computing environments. In: **IEEE International Conference on Cloud Computing (CLOUD)**. [S.l.]: IEEE, 2018. p. 442–450. Citado 3 vezes nas páginas 9, 17 e 20.
- MELL, Peter; GRANCE, Timothy. **The NIST Definition of Cloud Computing**. Gaithersburg, MD, USA, 2011. Disponível em: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>. Acesso em: 20 abr. 2025. Citado na página 12.
- MORRIS, Kief. **Infrastructure as Code**: Managing servers in the cloud. Sebastopol, CA, USA: O’Reilly Media, 2016. ISBN 9781491924358. Citado 2 vezes nas páginas 15 e 22.
- NAMASUDRA, Suyel *et al.* A comprehensive survey on cloud computing: Architecture, services, and deployment models. **ACM Computing Surveys**, Association for Computing

Machinery, v. 55, n. 1, p. 1–35, 2022. Disponível em: <https://dl.acm.org/doi/10.1145/3492348>. Citado na página 12.

PRESSMAN, Roger S. **Engenharia de Software - 7.ed.** McGraw Hill Brasil, 2009. ISBN 9788580550443. Disponível em: <https://books.google.com.br/books?id=y0rH9wuXe68C>. Citado na página 22.

ROBERTS, Mike. **Serverless Architectures**. Martin Fowler's Blog, 2018. Disponível em: <https://martinfowler.com/articles/serverless.html>. Acesso em: 20 abr. 2025. Citado 2 vezes nas páginas 7 e 13.

SOLANO, Antonio *et al.* Smart vending machines in the era of Internet of Things. **Future Generation Computer Systems**, Elsevier, v. 76, p. 215–220, 2017. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0167739X16304757>. Citado na página 11.

THETE, Shubham C. *et al.* From traditional to digital: An analysis of vending machine system evolution. **International Journal of Electrical Power System and Technology**, v. 10, n. 1, p. 1–7, 2024. Disponível em: <https://journalspub.com/wp-content/uploads/2024/09/1-7-From-Traditional-to-Digital-An-Analysis-of-Vending-Machine-System-Evolution-1-4.pdf>. Citado na página 11.

VendingExchange. **Do Vending Machines Need Wi-Fi?** 2025. VendingExchange Vending FAQs. Disponível em: <https://vendingexchange.com/vending-faqs/do-vending-machines-need-wifi>. Acesso em: 14 maio 2026. Citado 3 vezes nas páginas 7, 9 e 11.